Encapsulation in C# - Comprehensive Study Notes

What is Encapsulation?

Encapsulation is a fundamental principle of object-oriented programming that involves **bundling data (fields/variables) and methods (functions) together into a single unit (class)**, while **hiding the internal implementation details** from the outside world.[1][2][3][4][5]

Core Concepts:

- **Data Hiding**: Restricting direct access to internal data
- **Controlled Access**: Providing public interfaces (properties/methods) to interact with data
- **Information Hiding**: Concealing implementation details from external code
- **Bundling**: Grouping related data and behavior together[3][6][5]

Key Metaphor: Think of encapsulation like a capsule or medicine pill—the contents (data) are protected inside, and you interact with it only through defined interfaces.[4][1]

Real-World Example: A television remote control encapsulates complex circuitry. Users only interact through buttons (public interface), without knowing the internal wiring (hidden implementation).[7]

POCO Classes (Plain Old CLR Objects)

Definition

POCO stands for **Plain Old CLR Object**—simple C# classes that contain only properties and fields without any special framework dependencies or complex inheritance.[8]

Characteristics:

- Simple data containers
- No inheritance from framework base classes
- No special attributes or annotations
- Used primarily for data transfer and domain modeling
- Clean, lightweight, and testable

POCO Class Example

```
// Simple POCO class
public class Customer
{
    public int Id { get; set; }
    public string Name { get; set; }
    public string Email { get; set; }
    public DateTime DateOfBirth { get; set; }
    public string PhoneNumber { get; set; }
}

// Usage
Customer customer = new Customer
{
    Id = 1,
    Name = "John Doe",
    Email = "john@example.com",
    DateOfBirth = new DateTime(1990, 5, 15),
    PhoneNumber = "+1-555-0100"
};
```

Common Uses:

- Data Transfer Objects (DTOs)
- Domain models in Domain-Driven Design
- Entity classes for ORMs (Entity Framework)
- API request/response models

Properties: Getters and Setters

Understanding Properties

Properties are special members that provide a flexible mechanism to **read, write, or compute** the values of private fields. They combine aspects of both fields and methods.[9][10][11]

Components:

- **get accessor**: Retrieves the property value (getter)
- **set accessor**: Assigns a value to the property (setter)
- **Backing field**: Private field that stores the actual value

Basic Property Syntax

```
public class Person
{
    // Private backing field (hidden from outside)
    private string _name;

    // Public property with get and set
    public string Name
    {
        get { return _name; }
        set { _name = value; }  // 'value' is a keyword representing the assigned value
    }
}

// Usage
Person person = new Person();
person.Name = "Alice";           // Calls set accessor
Console.WriteLine(person.Name);  // Calls get accessor
```

Auto-Implemented Properties

C# provides a shorthand syntax where the compiler automatically creates the backing field.[12][10][11]

```
public class Product
{
    // Auto-implemented property - compiler creates hidden backing field
    public string Name { get; set; }
    public decimal Price { get; set; }
    public int Quantity { get; set; }
}

// Equivalent to:
/*
private string _name;
```

```
public string Name
{
    get { return _name; }
    set { _name = value; }
}
*/
```

Properties with Validation

Properties allow you to add validation logic to control how data is set.[2][3][4]

```
public class BankAccount
{
    private decimal _balance;

    public decimal Balance
    {
        get { return _balance; }
        set
        {
            if (value < 0)
                throw new ArgumentException("Balance cannot be negative");
            _balance = value;
        }
    }

    private int _age;

    public int Age
    {
        get { return _age; }
        set
        {
            if (value < 0 || value > 150)
                throw new ArgumentException("Invalid age");
            _age = value;
        }
    }
```

```
}
```

Benefits of Validation:

- Data integrity - prevents invalid states
- Business rule enforcement
- Input sanitization
- Error handling at the point of entry[2][3]

Read-Only Properties (Get Only)

Definition

A **read-only property** has only a **get accessor** and no **set accessor**. The property can only be read from outside the class, not modified.[10][12][9]

Traditional Read-Only Property

```
public class Circle
{
    private double _radius;

    public Circle(double radius)
    {
        _radius = radius;
    }

    // Read-only property - no set accessor
    public double Radius
    {
        get { return _radius; }
    }

    // Computed read-only property
    public double Area
    {
        get { return Math.PI * _radius * _radius; }
```

```
    }

    public double Circumference
    {
        get { return 2 * Math.PI * _radius; }
    }
}

// Usage
Circle circle = new Circle(5);
Console.WriteLine(circle.Area);       // ☑ Can read
// circle.Area = 100;                 // X Compile error - no setter
```

Auto-Implemented Read-Only Property (C# 6.0+)

```
public class Person
{
    // Can only be set in constructor or initializer
    public string Name { get; }
    public DateTime BirthDate { get; }

    public Person(string name, DateTime birthDate)
    {
        Name = name;
        BirthDate = birthDate;
    }
}

// Usage
Person person = new Person("Alice", new DateTime(1990, 1, 1));
Console.WriteLine(person.Name);  // ☑ Can read
// person.Name = "Bob";          // X Compile error
```

Read-Only with Private Setter

```
public class Employee
{
    // Can be set from within the class, but read-only from outside
    public string Name { get; private set; }
```

```
    public decimal Salary { get; private set; }

    public Employee(string name, decimal salary)
    {
        Name = name;
        Salary = salary;
    }

    // Internal method can modify private setter
    public void GiveRaise(decimal amount)
    {
        Salary += amount;  // ☑ Can modify within class
    }
}

// Usage
Employee emp = new Employee("John", 50000);
Console.WriteLine(emp.Salary);  // ☑ Can read
// emp.Salary = 60000;          // X Compile error - setter is private
emp.GiveRaise(5000);            // ☑ Modifies through method
```

Use Cases for Read-Only Properties:

- Computed/calculated values (Area, TotalPrice)
- Configuration values set once at initialization
- Identity fields (Id, CreatedDate)
- Derived properties based on other properties[13][12][9]

Write-Only Properties (Set Only)

Definition

A **write-only property** has only a **set accessor** and no **get accessor**. The property can be written to but not read from outside the class.[9][10]

Note: Write-only properties are **rare** in practice and generally discouraged, but they exist for specific scenarios.

Write-Only Property Example

```
public class SecureSystem
{
    private string _password;

    // Write-only property
    public string Password
    {
        set
        {
            // Validate and encrypt password
            if (string.IsNullOrWhiteSpace(value))
                throw new ArgumentException("Password cannot be empty");

            _password = EncryptPassword(value);
        }
    }

    private string EncryptPassword(string plainText)
    {
        // Encryption logic
        return Convert.ToBase64String(System.Text.Encoding.UTF8.GetBytes(plainText));
    }

    // Verification method instead of getter
    public bool VerifyPassword(string plainText)
    {
        return _password == EncryptPassword(plainText);
    }
}

// Usage
SecureSystem system = new SecureSystem();
system.Password = "MySecretPass123";  // ☑ Can write
// var pwd = system.Password;         // X Compile error - no getter
```

```
bool isValid = system.VerifyPassword("MySecretPass123");  // ☑ Verification instead
```

Use Cases for Write-Only Properties:

- Password fields (for security)
- Write-only configuration settings
- Data that should never be read back once set
- Logging sinks or output streams[10][9]

Why Rare?: Most scenarios require at least internal read access for testing, debugging, or verification. Consider using **private get** instead of completely omitting the getter.

Init-Only Properties (C# 9.0+)

Definition

Init-only properties are a C# 9.0 feature that allow properties to be set **only during object initialization** (constructor or object initializer), making them **immutable after creation**.[14][15][16][17]

Key Characteristics:

- Can be set during object initialization
- Cannot be modified after initialization completes
- Provides immutability without readonly fields
- Works with object initializers and constructors[16][18][14]

Init-Only Property Syntax

```
public class Person
{
    // Init-only properties
    public string FirstName { get; init; }
    public string LastName { get; init; }
    public DateTime BirthDate { get; init; }
    public int Age { get; init; }
}
```

```
// Usage
Person person = new Person
{
    FirstName = "Alice",
    LastName = "Smith",
    BirthDate = new DateTime(1990, 5, 15),
    Age = 34
};

Console.WriteLine(person.FirstName);  // ☑ Can read
// person.FirstName = "Bob";          // X Compile error - can't modify after init
```

Init with Constructor

```
public class Product
{
    public int Id { get; init; }
    public string Name { get; init; }
    public decimal Price { get; init; }

    public Product(int id, string name, decimal price)
    {
        Id = id;          // ☑ Can set in constructor
        Name = name;
        Price = price;
    }
}
```

Init with Custom Logic

Init accessors can contain validation logic and can modify **readonly fields**.[17][14][16]

```
public class Person
{
    private readonly string _firstName;

    public string FirstName
    {
```

```
        get => _firstName;
        init
        {
            if (string.IsNullOrWhiteSpace(value))
                throw new ArgumentException("FirstName cannot be empty");
            _firstName = value;  // ☑ Init can modify readonly fields
        }
    }

    public string LastName { get; init; }
}
```

Init vs Set vs Get-Only

Feature	`{ get; set; }`	`{ get; init; }`	`{ get; }`
Set in initializer	☑ Yes	☑ Yes	❌ No
Set in constructor	☑ Yes	☑ Yes	☑ Yes (field)
Modify after creation	☑ Yes	❌ No	❌ No
Immutability	❌ No	☑ Yes	☑ Yes
Flexibility	High	Medium	Low

Use Cases for Init-Only Properties:

- Data Transfer Objects (DTOs) that should be immutable
- Configuration objects
- Domain entities in Domain-Driven Design
- Records and value objects
- API response models[15][18][16]

Required Init Properties (C# 11+)

Combine `init` with `required` to enforce that properties must be set during initialization.[18]

```
public class Person
{
```

```
    public required string FirstName { get; init; }
    public required string LastName { get; init; }
    public int Age { get; init; }
}

// X Compile error - FirstName and LastName required
// Person person = new Person { Age = 30 };

// ☑ Correct
Person person = new Person
{
    FirstName = "Alice",
    LastName = "Smith",
    Age = 30
};
```

Sealed Classes (Final Classes)

Definition

A **sealed class** is a class that **cannot be inherited**. The `sealed` keyword prevents other classes from deriving from it.[19][20][21][22]

Note: In other languages like Java, sealed classes are called **final classes**.

Sealed Class Syntax

```
// Sealed class - cannot be inherited
public sealed class FinalConfiguration
{
    public string Setting { get; set; }
    public int MaxConnections { get; set; }

    public void Initialize()
    {
        Console.WriteLine("Initializing configuration");
    }
}
```

```
// X Compile error - cannot inherit from sealed class
// public class ExtendedConfiguration : FinalConfiguration
// {
// }
```

Real-World Example

```
public sealed class DatabaseConnection
{
    private string _connectionString;

    public DatabaseConnection(string connectionString)
    {
        _connectionString = connectionString;
    }

    public void Open()
    {
        Console.WriteLine($"Opening connection: {_connectionString}");
    }

    public void Close()
    {
        Console.WriteLine("Closing connection");
    }
}

// Can use the sealed class
DatabaseConnection conn = new DatabaseConnection("Server=localhost;");
conn.Open();
conn.Close();

// Cannot inherit
// class CustomConnection : DatabaseConnection { }  // X Error
```

Sealed Methods

You can also seal **individual methods** to prevent further overriding in multi-level inheritance.[20][21][22]

```
public class Animal
{
    public virtual void MakeSound()
    {
        Console.WriteLine("Animal sound");
    }
}

public class Dog : Animal
{
    // Sealed override - cannot be overridden further
    public sealed override void MakeSound()
    {
        Console.WriteLine("Woof!");
    }
}

public class Puppy : Dog
{
    // X Compile error - MakeSound is sealed in Dog
    // public override void MakeSound()
    // {
    //     Console.WriteLine("Puppy bark");
    // }
}
```

When to Use Sealed Classes

Advantages:

1. **Security**: Prevent malicious or unintended inheritance
2. **Performance**: Minor optimization - compiler can make devirtualization optimizations
3. **Design Control**: Finalize class behavior and prevent extensions
4. **Immutability Guarantee**: Ensure behavior cannot be changed through inheritance[23][19][20]

Use Cases:

- Utility classes (Math operations, String helpers)

- Security-sensitive classes
- Configuration classes
- Classes with critical business logic
- Framework classes like `String`, `Int32`, etc. are sealed[19][20]

When NOT to Use:

- When extensibility is desired
- In open-source libraries where users may want to extend
- When following Open/Closed Principle (open for extension)[24][23]

Sealed vs Private Classes

Feature	Sealed Class	Private Nested Class
Can be inherited	❌ No	N/A (only visible inside)
Visibility	Any access modifier	Only within containing class
Purpose	Prevent inheritance	Encapsulate helper logic
Scope	Any level	Nested only

```
public class OuterClass
{
    // Private nested class - encapsulation
    private class HelperClass
    {
        public void Help()
        {
            Console.WriteLine("Helping");
        }
    }

    public void UseHelper()
    {
        HelperClass helper = new HelperClass();
        helper.Help();
    }
}
```

```
// Cannot access or inherit HelperClass from here
// var helper = new OuterClass.HelperClass();  // X Error
```

Access Modifiers and Encapsulation

Access Modifiers Overview

Access modifiers control the **visibility and accessibility** of classes, methods, properties, and fields.[6][5][25][26]

Modifier	Accessible From	Common Use
private	Same class only	Internal implementation, backing fields
protected	Same class + derived classes	Members for inheritance
internal	Same assembly	Assembly-internal APIs
protected internal	Same assembly OR derived classes	Library extensibility
private protected	Same assembly AND derived classes	Restricted inheritance
public	Anywhere	Public APIs, interfaces

Private - Data Hiding

Most restrictive. Used for internal implementation details.[1][3][6]

```
public class BankAccount
{
    // Private fields - hidden from outside
    private decimal _balance;
    private string _accountNumber;
    private string _pin;

    public BankAccount(string accountNumber, string pin)
    {
        _accountNumber = accountNumber;
        _pin = pin;
```

```
        _balance = 0;
    }

    // Public interface to interact with private data
    public void Deposit(decimal amount)
    {
        if (amount > 0)
            _balance += amount;
    }

    public bool Withdraw(decimal amount, string pin)
    {
        if (pin == _pin && amount > 0 && _balance >= amount)
        {
            _balance -= amount;
            return true;
        }
        return false;
    }

    // Read-only access to balance
    public decimal GetBalance(string pin)
    {
        if (pin == _pin)
            return _balance;
        throw new UnauthorizedAccessException();
    }
}

// Usage
BankAccount account = new BankAccount("123456", "1234");
account.Deposit(1000);
// account._balance = 5000;  // X Error - private field
Console.WriteLine(account.GetBalance("1234"));  // ☑ Controlled access
```

Protected - Inheritance Access

Accessible in the same class and derived classes.[27][28]

```
public class Employee
{
    protected string _name;
    protected decimal _baseSalary;

    public Employee(string name, decimal baseSalary)
    {
        _name = name;
        _baseSalary = baseSalary;
    }

    public virtual decimal CalculateSalary()
    {
        return _baseSalary;
    }
}

public class Manager : Employee
{
    private decimal _bonus;

    public Manager(string name, decimal baseSalary, decimal bonus)
        : base(name, baseSalary)
    {
        _bonus = bonus;
    }

    public override decimal CalculateSalary()
    {
        // Can access protected members from base class
        return _baseSalary + _bonus;  // ☑ _baseSalary is protected
    }
}
```

Internal - Assembly Access

Accessible within the same assembly (project).[25][26][29]

```
// In Assembly A
internal class ConfigurationManager
{
    internal string ConnectionString { get; set; }

    internal void LoadConfig()
    {
        // Load configuration
    }
}

// Within same assembly - ☑ Can access
public class DataService
{
    private ConfigurationManager _config = new ConfigurationManager();
}

// In different assembly - X Cannot access ConfigurationManager
```

Public - Universal Access

Least restrictive. Accessible from anywhere.[5][6]

```
public class Calculator
{
    // Public methods - part of API contract
    public int Add(int a, int b)
    {
        return a + b;
    }

    public double Divide(double a, double b)
    {
        if (b == 0)
            throw new DivideByZeroException();
        return a / b;
    }
}
```

```
// Can be accessed from anywhere
Calculator calc = new Calculator();
int sum = calc.Add(5, 10);
```

Readonly Fields and Readonly Structs

Readonly Fields

A **readonly field** can only be assigned:

1. At declaration
2. In the constructor

After initialization, it cannot be changed.[30][31][32][13]

```
public class Configuration
{
    // Readonly field - can only be set at declaration or in constructor
    private readonly string _appName = "MyApp";
    private readonly int _maxRetries;
    private readonly DateTime _startupTime;

    public Configuration(int maxRetries)
    {
        _maxRetries = maxRetries;  // ☑ Can set in constructor
        _startupTime = DateTime.Now;
    }

    public void UpdateRetries(int newRetries)
    {
        // _maxRetries = newRetries;  // X Compile error - readonly
    }
}
```

Important: Readonly with Reference Types

Critical Understanding: With reference types, `readonly` prevents **reassignment of the reference**, but **NOT modification of the object's contents**.[31][32][33]

```
public class DataContainer
{
    // Readonly reference to a list
    private readonly List<string> _items = new List<string>();

    public void AddItem(string item)
    {
        _items.Add(item);  // ☑ Can modify list contents
    }

    public void ReplaceList()
    {
        // _items = new List<string>();  // X Error - cannot reassign
    }
}
```

To make the contents truly immutable, use immutable collections:

```
using System.Collections.Immutable;

public class DataContainer
{
    private readonly ImmutableList<string> _items = ImmutableList<string>.Empty;

    public void AddItem(string item)
    {
        // This creates a NEW immutable list, doesn't modify original
        _items = _items.Add(item);  // ☑ Returns new immutable list
    }
}
```

Readonly Structs (C# 7.2+)

A **readonly struct** enforces that **all fields and properties** are immutable.[34][35][36][33]

```
public readonly struct Point
{
    // All properties must be readonly or init-only
    public int X { get; }
    public int Y { get; }

    public Point(int x, int y)
    {
        X = x;
        Y = y;
    }

    // Methods cannot modify state
    public double DistanceFromOrigin()
    {
        return Math.Sqrt(X * X + Y * Y);
    }
}

// Usage
Point point = new Point(3, 4);
Console.WriteLine(point.DistanceFromOrigin());  // 5
// point.X = 10;  // X Error - readonly property
```

Benefits of Readonly Structs:

- **Immutability**: Guaranteed thread-safe
- **Performance**: Reduces defensive copies
- **Compiler enforcement**: Prevents accidental mutations
- **Clarity**: Documents intent[35][36][34]

Readonly vs Const

Feature	readonly	const
Set when	Runtime (constructor)	Compile time
Value type	Any type	Primitive types only

Static required	No	Implicitly static
Can change per instance	Yes	No (same for all)

```
public class MathHelper
{
    // const - compile time constant
    public const double PI = 3.14159;

    // readonly - runtime constant
    private readonly DateTime _createdAt;

    public MathHelper()
    {
        _createdAt = DateTime.Now;  // ☑ Set at runtime
    }
}
```

Advantages and Disadvantages of Encapsulation

Advantages

1. Data Protection and Security

Prevents unauthorized or accidental access to sensitive data.[37][3][5][7]

```
public class SecureUser
{
    private string _password;  // Hidden from outside

    public void SetPassword(string password)
    {
        // Validation and encryption before storing
        if (password.Length < 8)
            throw new ArgumentException("Password too short");
        _password = HashPassword(password);
    }
```

```
    public bool ValidatePassword(string password)
    {
        return _password == HashPassword(password);
    }
}
```

2. Flexibility and Maintainability

Internal implementation can change without affecting external code.[38][3][4][7]

```
public class Product
{
    private decimal _price;

    // Version 1: Simple property
    public decimal Price
    {
        get { return _price; }
        set { _price = value; }
    }

    // Version 2: Add tax calculation later - no external changes needed
    public decimal Price
    {
        get { return _price * 1.15m; }  // Now includes tax
        set { _price = value; }
    }
}
```

3. Controlled Access

Enforce business rules and validation.[3][7][2]

```
public class Employee
{
    private int _age;

    public int Age
```

```
    {
        get { return _age; }
        set
        {
            if (value < 18 || value > 65)
                throw new ArgumentException("Age must be between 18 and 65");
            _age = value;
        }
    }
}
```

4. Increased Code Reusability

Well-encapsulated classes are easier to reuse in different contexts.[7][3]

5. Easier Testing

Encapsulated code with clear interfaces is easier to test and mock.[37][7]

6. Reduces Complexity

Hides implementation details, presenting a simpler interface to users.[39][5][3]

Disadvantages

1. Increased Complexity

Too much encapsulation can make the codebase harder to understand.[39][7]

```
// Over-encapsulated - too many layers
public class OverEncapsulated
{
    private int _value;

    private int GetInternalValue() { return _value; }
    private void SetInternalValue(int val) { _value = val; }

    public int GetValue() { return GetInternalValue(); }
    public void SetValue(int val) { SetInternalValue(val); }
```

```
}
```

2. Development Overhead

Writing getters, setters, and validation adds initial development time.[7][39]

3. Performance Overhead (Minimal)

Method calls for getters/setters have tiny overhead compared to direct field access.[3][39][7]

Note: Modern compilers optimize this away in most cases through inlining.

4. Potential Misuse

Encapsulation can give false sense of security if not implemented properly.[39][7]

```
public class BadEncapsulation
{
    private List<string> _items = new List<string>();

    // Exposes internal mutable list - breaks encapsulation!
    public List<string> Items { get { return _items; } }
}

// External code can modify internal state
var obj = new BadEncapsulation();
obj.Items.Add("Hack");  // Modifies internal list directly!
```

Better approach:

```
public class GoodEncapsulation
{
    private List<string> _items = new List<string>();

    // Return read-only view
    public IReadOnlyList<string> Items { get { return _items.AsReadOnly(); } }

    // Or return a copy
    public List<string> GetItemsCopy()
    {
```

```
        return new List<string>(_items);
    }
}
```

5. Access Trade-offs

Sometimes making members private requires refactoring later when extension is needed.[7]

Best Practices for Encapsulation

1. Keep Fields Private

Always use private fields with public properties.[6][1][3]

```
// ☑ Good
public class Person
{
    private string _name;  // Private field

    public string Name     // Public property
    {
        get { return _name; }
        set { _name = value; }
    }
}

// X Bad
public class Person
{
    public string Name;  // Public field - no encapsulation
}
```

2. Use Properties Instead of Public Fields

Properties provide flexibility for future changes.[11][4][2]

```
// ☑ Good - can add logic later
public class Product
{
    public decimal Price { get; set; }
}

// Can evolve to:
public class Product
{
    private decimal _basePrice;

    public decimal Price
    {
        get { return _basePrice * 1.15m; }  // Add tax calculation
        set { _basePrice = value; }
    }
}
```

3. Validate in Setters

Enforce business rules and data integrity.[5][2][3]

```
public class Student
{
    private int _grade;

    public int Grade
    {
        get { return _grade; }
        set
        {
            if (value < 0 || value > 100)
                throw new ArgumentOutOfRangeException("Grade must be 0-100");
            _grade = value;
        }
    }
}
```

4. Return Immutable Collections

Don't expose mutable internal collections directly.[32][31]

```
public class Team
{
    private List<Player> _players = new List<Player>();

    // ☑ Good - return read-only view
    public IReadOnlyCollection<Player> Players => _players.AsReadOnly();

    // X Bad - exposes internal list
    // public List<Player> Players => _players;
}
```

5. Use Init for Immutability

Use init-only properties for immutable data objects.[15][16][18]

```
public class Order
{
    public int OrderId { get; init; }
    public DateTime OrderDate { get; init; }
    public decimal TotalAmount { get; init; }
}
```

6. Consider Sealed Classes

Seal classes that shouldn't be inherited.[20][23][19]

```
public sealed class Configuration
{
    // Prevent inheritance for security
}
```

Summary: Encapsulation Concepts Quick Reference

Property Types

Type	Syntax	Mutability	Use Case
Read-Write	`{ get; set; }`	Mutable	Standard properties
Read-Only	`{ get; }`	Immutable	Calculated values, constants
Write-Only	`{ set; }`	N/A	Passwords, sensitive data
Init-Only	`{ get; init; }`	Immutable after init	DTOs, immutable objects
Private Set	`{ get; private set; }`	Mutable internally	Controlled modification

Access Modifiers

Modifier	Visibility	Common Usage
private	Same class	Implementation details
protected	Class + derived	Inheritance members
internal	Same assembly	Internal APIs
public	Everywhere	Public APIs

Readonly Concepts

Feature	Immutability Level	Use Case
readonly field	Reference only	Configuration values
readonly struct	Complete	Value types, coordinates
const	Compile-time	Mathematical constants
init property	After initialization	Immutable DTOs

This comprehensive guide covers all essential aspects of encapsulation in C#, including POCO classes, properties (getters/setters, read-only, write-only, init-only), sealed classes, access modifiers, readonly concepts, advantages/disadvantages, and best practices.[4][14][16][31][32][1][2][9][10][6][5][15][3][7]

⁂

1. https://www.tutorialspoint.com/csharp/csharp_encapsulation.htm
2. https://dotnettutorials.net/lesson/encapsulation-csharp/
3. https://www.geeksforgeeks.org/c-sharp/encapsulation-in-c-sharp/
4. https://www.c-sharpcorner.com/article/encapsulation-in-C-Sharp/
5. https://www.scaler.com/topics/csharp/encapsulation-in-c-sharp/
6. https://www.codecademy.com/learn/learn-c-sharp/modules/learn-csharp-encapsulation/cheatsheet
7. https://dotnettutorials.net/lesson/real-time-examples-of-encapsulation-principle-in-csharp/
8. https://stackoverflow.com/questions/75218410/creating-poco-classes-for-complex-relationship-scenario-in-ef-core-7
9. https://learn.microsoft.com/en-us/dotnet/csharp/programming-guide/classes-and-structs/using-properties
10. https://www.geeksforgeeks.org/c-sharp/properties-in-c-sharp/
11. https://www.w3schools.com/cs/cs_properties.php
12. https://ironpdf.com/blog/net-help/csharp-getter-setter-guide/
13. https://www.pluralsight.com/resources/blog/guides/declare-and-initialize-read-only-auto-properties-in-c
14. https://developers.redhat.com/blog/2021/04/20/c-9-init-accessors-and-records
15. https://www.geeksforgeeks.org/c-sharp/init-only-properties-in-c-sharp/
16. https://learn.microsoft.com/en-us/dotnet/csharp/language-reference/keywords/init
17. https://www.thomasclaudiushuber.com/2020/08/25/c-9-0-init-only-properties/
18. https://antondevtips.com/blog/csharp-init-only-and-required-properties
19. https://ironpdf.com/blog/net-help/csharp-sealed-class-guide/
20. https://www.geeksforgeeks.org/c-sharp/sealed-class-in-c-sharp/
21. https://www.programiz.com/csharp-programming/sealed-class
22. https://learn.microsoft.com/en-us/dotnet/csharp/language-reference/keywords/sealed
23. https://stackoverflow.com/questions/6389417/should-i-recommend-sealing-classes-by-default

24. https://enlear.academy/private-vs-sealed-classes-in-c-91147c3f8bfa

25. https://www.w3schools.com/cs/cs_access_modifiers.php

26. https://learn.microsoft.com/en-us/dotnet/csharp/programming-guide/classes-and-structs/access-modifiers

27. https://ironpdf.com/blog/net-help/csharp-protected/

28. https://www.c-sharpcorner.com/blogs/use-of-protected-methods-in-c-sharp

29. https://www.tutlane.com/tutorial/csharp/csharp-access-modifiers-public-private-protected-internal

30. https://www.reddit.com/r/csharp/comments/hefuiy/private_readonly_fields_vs_private_getteronly/

31. https://www.linkedin.com/pulse/when-use-readonly-c-key-safer-code-andre-baltieri-x7u8f

32. https://www.c-sharpcorner.com/article/why-readonly-doesnt-make-objects-immutable-in-c-sharp/

33. https://steven-giesel.com/blogPost/2a52cd8d-b3b2-42e4-87e8-d6dc14147ddb/struct-vs-readonly-struct-vs-ref-struct-vs-record-struct

34. https://www.geeksforgeeks.org/c-sharp/readonly-structs-and-ref-readonly-in-c/

35. https://stackoverflow.com/questions/6063212/does-using-public-readonly-fields-for-immutable-structs-work

36. https://csharp-evolution.com/7.2/readonly-struct

37. https://embarkingonvoyage.com/blog/technologies/why-c-encapsulation-is-critical-for-enterprise-grade-software-architecture/

38. https://giannisakritidis.com/blog/Encapsulation/

39. https://www.upgrad.com/blog/encapsulation-in-oops/

40. https://www.youtube.com/playlist?list=PLyVQKSvkg_jojSrprWHQtx9VHAjFJ-yRz

41. https://dotnettutorials.net/course/csharp-dot-net-tutorials/

42. https://www.youtube.com/playlist?list=PLZyQU-WOzZF2g5PCSHfhHvoX7BnCUn3Md

43. https://www.youtube.com/playlist?list=PLT5F0JLnkk3iUkeTbzwORo3DiUY1og5cq

44. https://www.youtube.com/playlist?list=PLdHN14J7CHtZJg2EZKq4Y6lVaALYzGWcu

45. https://www.youtube.com/playlist?list=PLuOs64mR1bYcD9Fz6LtCXam-99IprxGmE

46. https://www.youtube.com/playlist?list=PLkX6P-stVWK-giKmWJFf_CWqbCMUnDPhD

47. https://www.youtube.com/playlist?list=PLsQKsFc4KZM2OgBPD5QFkj4OIt-rSHocn

48. https://www.youtube.com/playlist?list=PL7J-zw69jRon6Y0537yINE-PreLfHKxmi

49. https://www.youtube.com/playlist?list=PLnJCXWv58vgqWf4gdLx_tLeKjGgX686Mg

50. https://www.youtube.com/playlist?list=PLINzwvqnPkcyn-IaCFVhNUvndUtQlOq2u

51. https://stackoverflow.com/questions/2691090/readonly-property-setter

52. https://www.youtube.com/playlist?list=PL0D2PELlR1N3Wc63n9yZllilujT6fsYGy

53. https://stackoverflow.com/questions/64749277/in-c9-how-do-init-only-properties-differ-from-read-only-properties

54. https://pwskills.com/blog/encapsulation-in-c-sharp/

55. https://www.jetbrains.com/help/resharper/PropertyCanBeMadeInitOnly.Global.html

56. https://learn.microsoft.com/en-us/dotnet/csharp/language-reference/builtin-types/struct